

08 — Reproducibility, Software, and Artifacts

SEION-KGR v26 MAX

Campaign ID: `gate12-closeout-2026-08-01`

Canonical commit: `6af3c35271ae2ffab41ecba2aad098d1988fdc0c`

Branch: `campaign/gate12-closeout`

Date: `2026-08-01`

Purpose: environment, hardware, test matrix, CI, parity checks, run identity, regeneration commands.

Contents

1	Environment and hardware	1
2	Test matrix	1
3	Continuous integration	1
4	Reference-vs-optimized and precision parity	1
5	Run identity	2
6	Regeneration commands	2
7	Limitations of this reproduction	2

1 Environment and hardware

- Python 3.12.10 (local); CI pins Python 3.11 ([.github/workflows/kgr-ci.yml](#)).
- PyTorch 2.12.0.dev20260408+cu128.
- GPU: NVIDIA RTX PRO 5000 Blackwell Generation Laptop GPU, 24463 MiB VRAM (real, used for Phase B5's BF16/FP32 parity tests and the Tier 0 A0 base-expert screening runs).
- OS: Windows 11 Pro, MiKTeX (pdflatex/xelatex/latexmk) for this package.

2 Test matrix

120 tests in `tests/kgr/` at the end of this campaign (66 at the canonical commit `6af3c35271ae2ffab41ecba2aad098d1988fdc0c`, 54 added across Phases B2–B5 and D):

File	Adds
<code>test_selector.py</code>	14 — learned selector (Phase B2)
<code>test_structural_kernel.py</code>	19 — E8/matched controls (Phase B3)
<code>test_certification.py</code>	15 — certification chain (Phase B4)
<code>test_perf_and_parity.py</code>	4 — reference/optimized + BF16/FP32 parity (Phase B5)
<code>test_campaign_negative_controls.py</code>	2 — integration-level negative controls (Phase D)

Run: `python -m pytest tests/kgr -q`. All 120 pass at the campaign's final commit on this branch.

3 Continuous integration

`.github/workflows/kgr-ci.yml`, path-filtered on `seion_kgr/**` and related files: installs `[test,research]` on Python 3.11, runs the FP64 oracle self-test, the trainer self-test (16 architecture combinations), the full pytest battery with JUnit output, a tiny synthetic file-based

end-to-end smoke test, and a determinism spot-check (two oracle invocations, same seed, asserted byte-identical). Real, executed run on branch `campaign/gate12-closeout`: job `seion-kgr-ci`, run id 30726372321, **success**, 1m41s. Three other pre-existing workflows on the same push also passed (`fast-cpu`, `seion-canonical-v4`, `security-and-quality`); one pre-existing, unrelated workflow (`spectral-v18-cpu`) failed on a known-flaky floating-point-boundary test in `spectral/certification_v18/tests/test_block_m.py` ($1.49 \times 10^{-8} < 10^{-8}$) — not touched by this campaign, not part of `seion_kgr`.

4 Reference-vs-optimized and precision parity

- `seion_kgr_reference_fp64.CPTernaryLaw` vs. `seion_kgr.kernels.CPTernaryLaw`: identical copied weights, identical output to $\text{atol} = 10^{-4}$, across 6 random cases.
- BF16 on the real GPU vs. FP32 on CPU: relative error $< 5\%$ (declared tolerance, appropriate for BF16's ~ 3 decimal digits of precision).
- FP32 CPU vs. FP32 GPU (device-placement check, tighter): $\text{atol} = 10^{-5}$.

5 Run identity

Every run directory carries `configuration_id` (SHA-256 of the resolved config, excluding run-control fields: `seed`, `out_dir`, `resume`, `epochs`, `eval cadence/subset`), a fresh `execution_id` per invocation, and `parent_execution_id` when resumed. A resume whose `configuration_id` does not match its parent checkpoint is rejected with a clear error before any state is loaded — live-verified, including catching and fixing two real bugs during this campaign: `epochs` was initially (wrongly) part of the configuration hash, which broke the ordinary "resume for more epochs" case; and the mismatch check initially ran *after* `load_state_dict`, surfacing a confusing raw PyTorch shape-mismatch error instead of the intended message on a genuine architecture-mismatch resume. Both fixed and reverified with real CLI runs, not just unit tests.

An out-dir reuse guard rejects a non-empty `-out_dir` unless `-resume` is passed — live-verified: a fresh run into an occupied directory raises `FileExistsError` with an explanatory message.

6 Regeneration commands

```
# Full test suite
python -m pytest tests/kgr -q

# Both self-tests
python seion_kgr_reference_fp64.py --self_test
python seion_kgr_v26_train.py --self_test

# Tier 0 base-expert selection (FB15K-237, 1 epoch, GPU)
for BASE in complex distmult tucker cp; do
    python seion_kgr_v26_train.py --train data/FB15K-237/train.txt \
        --valid data/FB15K-237/valid.txt --test data/FB15K-237/test.txt \
        --out_dir runs/base_selection_${BASE} --dim 64 --base_expert ${BASE} \
        --epochs 1 --batch_size 1024 --neg_k 64 --eval_max_queries 200 \
        --entity_block_eval 4096 --seed 1
done
```

7 Limitations of this reproduction

- `E8_Exact_v18_2/f_E8.npy` (59MB) is not committed to git (same convention as `data/`); `E8_exact`-variant tests and runs are skipped/unavailable without it placed locally.
- `data/` (FB15K-237, WN18RR, CoDEx-M, DB100K+, YAGO3-10) is not committed; must be fetched/placed locally.
- `runs/` and `campaigns/*/tier*/` are gitignored (checkpoints, large binaries) — this package's `manifests/` directory carries the compact JSON summaries and hashes needed to audit what ran, not the checkpoints themselves.